

NamingContest.com — Pre-Launch Review Brief

Scope, priorities and deliverable for the pre-launch security review

For the senior developer engaged to review the platform before launch. This is your scope, your priorities, and what to hand back.

Read this first, then the Developer Handbook (how the system works) and the Going Live guide (setup and the launch procedure). Both are in [docs/](#).

The engagement in one line

An independent security and code review of a complete, in-production platform before it opens to the public and takes real money. Find what a single build pass missed. Report it ranked by severity, each with a fix.

What you're reviewing

A paid naming-contest platform, live behind a beta gate. React 18 + Vite SPA, Supabase (Postgres with row-level security, Deno edge functions, pg_cron), Stripe Payment Intents plus a webhook, Resend email, Vercel. Money is real once test keys become live keys — nothing else changes. So the security bar is a payments bar, not a prototype bar.

Priorities, highest first

1. Payment integrity. The whole business is here. - Can any path mark a contest paid or live without a verified Stripe payment? (The intended guards are RLS + triggers [0017](#) price, [0021](#) paid/status columns, [0022](#) voter-tier cap — verify they hold and that nothing routes around them.) - Can the amount, tier, or price be manipulated from the client? - The browser confirmation path and the [stripe-webhook](#) both complete a launch. Stress the idempotency: double receipts, race conditions, a webhook arriving before the browser, replayed events. - Advise on the test-to-live switch (Going Live guide §4) — anything that bites specifically at the mode boundary.

2. Data access and RLS. One policy hole exposes every user's data. - Every table's RLS. Can a participant read another's identity or votes? Can vote tallies leak before a contest closes? ([0018](#) [get_ballot](#) and [0019](#) are the intended controls.) - The SECURITY DEFINER functions — do any over-

return? - Edge function auth: three run `--no-verify-jwt` and authenticate themselves (rate limits, the notify secret, the Stripe signature). Are those self-checks sound?

3. Auth and session handling. The single largest bug source in this codebase's history, in two shapes: a localStorage "guest blob" standing in for a real session, and effects querying RLS-protected data before auth resolves. Both were fixed where found. **Probe for survivors** — this is where to spend disproportionate time.

4. Input handling. XSS/injection via user-supplied contest names, submissions, contact-form fields. Email HTML escaping (`esc()` in `_shared/email.ts`) — is anything interpolated raw?

5. Everything else. Rate limiting (deliberately fail-open — agree or challenge that call), secrets handling, CORS, error leakage, dependency CVEs, GDPR posture (the privacy policy and terms are live pages).

Where to start, concretely

- `DEVELOPER-HANDBOOK.md` §6 lists every migration and the server-enforced invariants — that's the security model in one place.
- `DEVELOPER-HANDBOOK.md` §13 is an honest known-gaps list. Confirm or dismiss each; they are leads, not the full picture.
- The migrations `supabase/migrations/0001-0023` are the source of truth for all database logic — there is no ORM.
- The code is heavily commented, including the *why* behind fixed bugs. Those comments are review material, not just documentation.

What is NOT in scope

Feature work, redesign, the mock/demo scaffolding still in the tree (unrouted, documented), and performance beyond flagging anything egregious. If you spot product-level improvements, note them separately from the security findings.

Access you'll be given

Repo access, a staging environment, and read access to the dashboards needed to trace the payment and email paths. Request anything missing.

Deliverable

A written report:

1. **Findings, ranked by severity** (critical / high / medium / low). Each: what it is, how to reproduce or where in the code, the impact, and a recommended fix. A concrete repro beats a description.
2. **A go / no-go for launch** — is anything critical open?
3. **The test-to-live switch** — confirmed safe, or blockers.

Please also flag anything you'd want fixed but couldn't fully verify, so nothing sits silently in the "maybe" column.

Recommended hardening — the builder's own list

Written by the developer who built the platform. These are known directions to strengthen it, not defects — offered so you spend your time finding what's unknown rather than rediscovering what's already visible. Weigh them, add to them, and fold anything you agree with into the report so the client gets one ranked list.

Security

- **Beta gate is client-side only.** `VITE_BETA_PASSWORD` ships in the JS bundle — obscurity, fine for a soft beta, not access control. If the private period ever needs to be real, move it server-side (a Supabase Auth allowlist, or a Vercel edge middleware check).
- **Rate limiting fails open** by design (a limiter error lets the request through). Correct for these low-stakes endpoints, but confirm the trade and consider a hard cap for a genuine flood.
- **DMARC is `p=none`** (monitoring only). Tighten to `quarantine` then `reject` over a few weeks of clean reports — raises deliverability and closes spoofing of the sending domain.
- **No Stripe webhook signature-replay window check** beyond the library's own. Confirm `constructEventAsync` tolerance and whether replay past the idempotency guard is possible.
- **Secrets rotation has no schedule.** Document and rotate on a cadence; `NOTIFY_SECRET` lives in two places (Vault + function secret) and both must move together.
- **Content Security Policy / security headers** are Vercel defaults. Adding a CSP, HSTS, and `X-Content-Type-Options` via `vercel.json` is cheap defence-in-depth.

Backend and data

- **The notify trigger hardcodes the production project ref** in its function URL (see Going Live §5). Read it from a GUC/Vault value instead, so a fresh environment can't accidentally call production.
- **No Stripe webhook for `payment_intent.payment_failed` or `charge.refunded`** — the system never learns about failures or refunds out-of-band. Worth wiring if refund/dispute volume grows.
- **Payment verification is pull-based** in `confirm-launch`; the webhook is the backstop. Solid, but a reviewer may prefer the webhook as the primary and the browser call as the optimisation. Worth a view.
- **`net._http_response` grows unbounded** — `pg_net` logs every trigger call. Add a prune job like the rate-limit one (`0020`).

- **No soft-delete / audit trail** on contests or payments. For a system taking money, an append-only record of state changes aids disputes.

Reliability and maintainability

- **No automated tests.** The build is the only gate. Even a thin integration suite over the payment path and the RLS policies would catch the regression classes that bit this build.
- **~150 `mockContest` branches** thread demo scaffolding through real pages (14 files). Unrouted and documented, but removing it is a real refactor that shrinks the surface a reviewer and future dev must reason about.
- **Single ~1.7 MB JS bundle**, no code-splitting. Route-level lazy imports would help mobile first-load materially.
- **Shared error-handling / retry** across the edge functions is copy-pasted, not factored. A shared wrapper would make behaviour uniform.
- **Legacy v1 page components remain** in `src/pages/` (unrouted). Safe to delete in a cleanup pass.

Effort and logistics

Estimated 10–20 hours. Questions to the point of contact throughout — a mid-review question is cheaper than a wrong assumption in the report. An optional follow-up engagement to implement the fixes can be scoped once the findings are in.